SENATI
Curso: Inteligencia Artificial

Guía 01

Operaciones con las librerías NumPy y Pandas

Manipulación y análisis de datos con Python en Visual Studio Code

Dato	Detalle
Instructor	Rembrandt Ubalde Enríquez
Estudiante	_____
Fecha	____ / ____ / 2026
Duración estimada	4 horas pedagógicas
Herramientas	Python 3, Visual Studio Code, NumPy, Pandas, Matplotlib

Guía práctica paso a paso con Visual Studio Code

Contenido

- Presentación de la guía
- Parte 1. Preparación del entorno en Visual Studio Code
- Parte 2. Operaciones con NumPy
- Parte 3. Operaciones con Pandas
- Parte 4. Visualización, cuadernos y depuración
- Parte 5. Errores frecuentes y cómo resolverlos
- Parte 6. Ejercicios propuestos
- Lista de cotejo (autoevaluación)

Presentación de la guía

Objetivo general

Realizar operaciones de creación, cálculo, limpieza, transformación y resumen de datos usando las librerías **NumPy** y **Pandas** en Visual Studio Code, como base para proyectos de inteligencia artificial.

Objetivos específicos

- Configurar un proyecto de Python con entorno virtual en VS Code.
- Crear arreglos de NumPy y aplicar operaciones vectorizadas, estadísticas y matriciales.
- Crear Series y DataFrames de Pandas y explorarlos.
- Seleccionar, filtrar, ordenar, agrupar y crear columnas calculadas.
- Leer y limpiar un archivo CSV, tratar valores faltantes y exportar un reporte a Excel.
- Visualizar resultados con un gráfico y depurar el código en VS Code.

¿Por qué NumPy y Pandas en inteligencia artificial?

Todo modelo de IA aprende a partir de datos. Antes de entrenar un modelo hay que **cargar, limpiar y transformar** esos datos, y ese trabajo se hace con Pandas. Internamente, los modelos trabajan con matrices de números, que se manejan con NumPy. Por eso estas dos librerías son la puerta de entrada a Scikit-learn, PyTorch y cualquier otra herramienta de IA.

Librería	Estructura principal	Uso típico
NumPy	`ndarray` (arreglo de n dimensiones)	Cálculos numéricos rápidos, álgebra lineal, números aleatorios, base de otras librerías.
Pandas	`Series` (columna) y `DataFrame` (tabla)	Leer CSV/Excel, limpiar datos, filtrar, agrupar, crear reportes.

Requisitos previos

- Computadora con Windows 10/11, Linux o macOS y conexión a Internet.
- Conocimientos básicos de Python: variables, listas, diccionarios, bucles y funciones.

Parte 1. Preparación del entorno en Visual Studio Code

En esta parte dejarás listo tu equipo para programar en Python dentro de Visual Studio Code (VS Code). Si ya lo hiciste en una guía anterior, revisa rápidamente cada paso y continúa desde el **Paso 4** creando una carpeta nueva para esta guía.

Paso 1. Instalar Python

1. Ingresa a **python.org/downloads** y descarga la versión estable más reciente de Python 3 (3.11 o superior).
2. Ejecuta el instalador. En la primera pantalla **marca la casilla "Add python.exe to PATH"** y luego pulsa **Install Now**.
3. Al terminar, abre el **Símbolo del sistema** (cmd) y verifica la instalación con los comandos de abajo.

Símbolo del sistema (Windows)

```
python --version
pip --version
```

Resultado esperado en la terminal

```
Python 3.12.x
pip 24.x from C:\Users\...\site-packages\pip (python 3.12)
```

Si el comando python no funciona

Mensaje "No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...": Python no está instalado o no está en el PATH, y Windows redirige el comando a la Store. Para solucionarlo:
1. Instala Python desde **python.org** marcando **Add python.exe to PATH**. Sin permisos de administrador, desmarca **Use admin privileges when installing py.exe**. También puedes usar en la terminal: `winget install Python.Python.3.12`
2. Abre **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** y desactiva **python.exe** y **python3.exe** (Instalador de aplicación).
3. **Cierra VS Code por completo y vuelve a abrirlo**: una terminal ya abierta conserva el PATH anterior.
4. Verifica con `python --version` o con `py --version`.

Mensaje "python no se reconoce como un comando interno": Python se instaló sin agregarse al PATH. Vuelve a ejecutar el instalador, elige **Modify** y activa **Add Python to environment variables**.

Paso 2. Instalar Visual Studio Code y sus extensiones

1. Descarga VS Code desde **code.visualstudio.com** e instálalo. Durante la instalación marca **"Agregar la acción Abrir con Code"** y **"Agregar a PATH"**.
2. Abre VS Code y entra al panel de **Extensiones** (ícono de cuadritos en la barra izquierda o `Ctrl + Shift + X`).
3. Busca e instala la extensión **Python** (publicada por Microsoft). Se instalará también **Pylance**, que da autocompletado y detección de errores.
4. Instala la extensión **Jupyter** (Microsoft) para trabajar con cuadernos `.ipynb` dentro de VS Code.

5. Opcional: instala **Spanish Language Pack for Visual Studio Code** si prefieres los menús en español.

Paso 3. Conocer la interfaz de VS Code

Zona de VS Code	Para qué sirve
Explorador (`Ctrl + Shift + E`)	Muestra las carpetas y archivos del proyecto. Desde aquí creas archivos nuevos.
Editor	Zona central donde escribes el código. Puedes tener varias pestañas abiertas.
Terminal integrada (`Ctrl + ñ` o menú **Ver > Terminal**)	Ejecuta comandos (pip, python) sin salir de VS Code.
Paleta de comandos (`Ctrl + Shift + P`)	Busca cualquier acción de VS Code escribiendo su nombre.
Barra de estado (abajo)	Muestra el intérprete de Python activo, errores y advertencias.
Botón ▶ Run Python File (arriba a la derecha)	Ejecuta el archivo `.py` abierto y muestra el resultado en la terminal.

Paso 4. Crear la carpeta del proyecto

1. En el Explorador de Windows crea la carpeta **IA_Guia01_NumPy_Pandas** (por ejemplo, en Documentos).
2. En VS Code ve a **Archivo > Abrir carpeta...** y selecciona la carpeta creada.
3. Si aparece el mensaje **"¿Confía en los autores de los archivos de esta carpeta?"**, pulsa **Sí, confío en los autores**.

Paso 5. Crear y activar un entorno virtual

Un **entorno virtual** es una instalación de Python aislada solo para este proyecto. Evita conflictos entre versiones de librerías de distintos proyectos.

1. Abre la terminal integrada con **Ver > Terminal** (o `Ctrl + ñ`).
2. Escribe el comando para crear el entorno virtual llamado `venv`.
3. Actívalo. Sabrás que está activo porque al inicio de la línea aparece **(venv)**.

Terminal de VS Code

```
# Crear el entorno virtual
python -m venv venv

# Activarlo en Windows (PowerShell)
.\venv\Scripts\Activate.ps1

# Activarlo en Windows (cmd)
venv\Scripts\activate

# Activarlo en Linux / macOS
```

```
source venv/bin/activate
```

> **Importante**
> Si PowerShell muestra **"la ejecución de scripts está deshabilitada en este sistema"**, ejecuta una sola vez:
> `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` y responde **S** (Sí). Luego vuelve a activar el entorno.

Paso 6. Seleccionar el intérprete de Python en VS Code

1. Presiona `Ctrl + Shift + P` y escribe **Python: Select Interpreter**.
2. Elige la opción que contiene **('venv': venv)** y la ruta `.\venv\Scripts\python.exe`.
3. Comprueba en la barra de estado (parte inferior derecha) que aparezca la versión de Python con **('venv')**.

Paso 7. Instalar las librerías de la guía

Con el entorno virtual activo, instala las librerías desde la terminal integrada:

Terminal de VS Code

```
python -m pip install --upgrade pip
pip install numpy pandas matplotlib openpyxl
```

Verifica que la instalación fue correcta:

Terminal de VS Code

```
python -c "import numpy, pandas; print(numpy.__version__, pandas.__version__)"
```

> **Sugerencia**
> Guarda la lista de librerías de tu proyecto con `pip freeze > requirements.txt`. Así un compañero puede instalar lo mismo con `pip install -r requirements.txt`.

Paso 8. Cómo crear y ejecutar cada archivo de esta guía

1. En el Explorador de VS Code pulsa el ícono **Nuevo archivo** (o `Ctrl + N` y luego `Ctrl + S`).
2. Escribe el nombre indicado en la guía con extensión `.py` (por ejemplo `01_ejemplo.py`) y presiona Enter.
3. Copia el código, guárdalo con `Ctrl + S` y ejecútalo con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_archivo.py` en la terminal.
4. Compara lo que aparece en la terminal con el recuadro verde **"Resultado esperado"**.
5. Para ejecutar solo algunas líneas, selecciónalas y presiona `Shift + Enter`: se ejecutarán en la ventana interactiva de Python.

Parte 2. Operaciones con NumPy

Paso 9. Crear arreglos con NumPy

Crea el archivo **01_numpy_basico.py**. Por convención, NumPy se importa con el alias `np`.

Archivo: 01_numpy_basico.py

```
# 01_numpy_basico.py - Creación de arreglos con NumPy
import numpy as np

print("Versión de NumPy:", np.__version__)

# 1. Crear un arreglo a partir de una lista
notas = np.array([14, 16, 11, 18, 13])
print("Arreglo de notas:", notas)
print("Tipo de dato:", notas.dtype)
print("Dimensiones (shape):", notas.shape)

# 2. Arreglos especiales
ceros = np.zeros(4)
unos = np.ones((2, 3))
rango = np.arange(0, 10, 2)        # inicio, fin (excluido), paso
lineal = np.linspace(0, 1, 5)      # 5 valores entre 0 y 1
print("Ceros:", ceros)
print("Unos (2x3):\n", unos)
print("arange:", rango)
print("linspace:", lineal)

# 3. Arreglo de 2 dimensiones (matriz) y reshape
matriz = np.arange(1, 13).reshape(3, 4)
print("Matriz 3x4:\n", matriz)
print("Número de dimensiones:", matriz.ndim, "| Total de elementos:", matriz.size)
```

Resultado esperado en la terminal

```
Versión de NumPy: 2.5.3
Arreglo de notas: [14 16 11 18 13]
Tipo de dato: int64
Dimensiones (shape): (5,)
Ceros: [0. 0. 0. 0.]
Unos (2x3):
 [[1. 1. 1.]
 [1. 1. 1.]]
arange: [0 2 4 6 8]
linspace: [0.   0.25 0.5  0.75 1.  ]
Matriz 3x4:
 [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Número de dimensiones: 2 | Total de elementos: 12
```

Explicación

shape indica el tamaño de cada dimensión: `(5,)` es un vector de 5 elementos y `(3, 4)` una matriz de 3 filas y 4 columnas.
dtype es el tipo de dato de todos los elementos (int64, float64...). Un arreglo de NumPy solo guarda un tipo de dato.

reshape reorganiza los datos sin cambiarlos; el total de elementos debe coincidir (12 = 3 × 4).

Paso 10. Operaciones vectorizadas, estadísticas y matrices

Crea **02_numpy_operaciones.py**. La gran ventaja de NumPy es que opera sobre todo el arreglo a la vez, **sin escribir bucles for**, lo que es mucho más rápido.

Archivo: 02_numpy_operaciones.py

```
# 02_numpy_operaciones.py - Operaciones vectorizadas y estadísticas
import numpy as np

a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

# 1. Operaciones elemento a elemento (sin bucles for)
print("Suma:", a + b)
print("Resta:", a - b)
print("Multiplicación:", a * b)
print("División:", a / b)
print("Potencia:", b ** 2)

# 2. Broadcasting: operar un arreglo con un escalar
print("a + 5 =", a + 5)

# 3. Estadísticas básicas
notas = np.array([14, 16, 11, 18, 13, 9, 15])
print("Promedio:", notas.mean().round(2))
print("Máximo:", notas.max(), "| Mínimo:", notas.min())
print("Desviación estándar:", notas.std().round(2))
print("Mediana:", np.median(notas))

# 4. Indexación, slicing y máscaras booleanas
print("Primera nota:", notas[0], "| Última nota:", notas[-1])
print("Notas de la 2 a la 4:", notas[1:4])
aprobados = notas[notas >= 13]
print("Notas aprobatorias (>=13):", aprobados)

# 5. Operaciones con matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("Producto matricial A @ B:\n", A @ B)
print("Transpuesta de A:\n", A.T)
print("Suma por columnas de A:", A.sum(axis=0))
print("Suma por filas de A:", A.sum(axis=1))

# 6. Números aleatorios reproducibles
rng = np.random.default_rng(seed=42)
aleatorios = rng.integers(0, 21, size=5)
print("5 notas aleatorias:", aleatorios)
```

Resultado esperado en la terminal

```
Suma: [11 22 33 44]
Resta: [ 9 18 27 36]
Multiplicación: [ 10  40  90 160]
División: [10. 10. 10. 10.]
```

```
Potencia: [ 1  4  9 16]
a + 5 = [15 25 35 45]
Promedio: 13.71
Máximo: 18 | Mínimo: 9
Desviación estándar: 2.81
Mediana: 14.0
Primera nota: 14 | Última nota: 15
Notas de la 2 a la 4: [16 11 18]
Notas aprobatorias (>=13): [14 16 18 13 15]
Producto matricial A @ B:
 [[19 22]
 [43 50]]
Transpuesta de A:
 [[1 3]
 [2 4]]
Suma por columnas de A: [4 6]
Suma por filas de A: [3 7]
5 notas aleatorias: [ 1 16 13  9  9]
```

Explicación

Broadcasting: al sumar `a + 5`, NumPy “extiende” el 5 a todos los elementos.
Máscara booleana: `notas >= 13` genera un arreglo de True/False; `notas[notas >= 13]` devuelve solo los True.
axis=0 opera por columnas (de arriba hacia abajo) y **axis=1** por filas (de izquierda a derecha).
`A @ B` es el producto matricial (filas × columnas); `A * B` sería multiplicación elemento a elemento.
`default_rng(seed=42)` hace que los números aleatorios sean siempre los mismos: útil para repetir experimentos de IA.

Parte 3. Operaciones con Pandas

Paso 11. Crear Series y DataFrames

Crea **03_pandas_dataframe.py**. Pandas se importa con el alias `pd`.

Archivo: 03_pandas_dataframe.py

```
# 03_pandas_dataframe.py - Series y DataFrames
import pandas as pd

print("Versión de Pandas:", pd.__version__)

# 1. Series: una columna con índice
edades = pd.Series([19, 21, 20, 23], index=["Ana", "Luis", "Rosa", "Pedro"])
print(edades)
print("Edad de Rosa:", edades["Rosa"])

# 2. DataFrame: tabla con filas y columnas
datos = {
    "alumno": ["Ana", "Luis", "Rosa", "Pedro", "Carla", "Jorge"],
    "carrera": ["Software", "Redes", "Software", "Redes", "Software", "Redes"],
    "nota1": [15, 12, 18, 10, 14, 16],
    "nota2": [17, 11, 16, 13, 12, 15],
}
df = pd.DataFrame(datos)
print(df)

# 3. Explorar el DataFrame
print("Filas y columnas:", df.shape)
print("Columnas:", list(df.columns))
print(df.head(3))
df.info()
print(df.describe())
```

Resultado esperado en la terminal (resumen)

```
Versión de Pandas: 3.0.5
Ana      19
Luis     21
Rosa     20
Pedro    23
dtype: int64
Edad de Rosa: 20
  alumno   carrera  nota1  nota2
0    Ana  Software     15     17
1   Luis     Redes     12     11
2   Rosa  Software     18     16
3  Pedro     Redes     10     13
4  Carla  Software     14     12
5  Jorge     Redes     16     15
Filas y columnas: (6, 4)
Columnas: ['alumno', 'carrera', 'nota1', 'nota2']
  alumno   carrera  nota1  nota2
0    Ana  Software     15     17
1   Luis     Redes     12     11
2   Rosa  Software     18     16
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
```

```
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   alumno   6 non-null      str
 1   carrera  6 non-null      str
 2   nota1    6 non-null      int64
 3   nota2    6 non-null      int64
dtypes: int64(2), str(2)
memory usage: 324.0 bytes
           nota1      nota2
count   6.000000   6.000000
mean   14.166667  14.000000
std     2.857738   2.366432
min    10.000000  11.000000
25%    12.500000  12.250000
50%    14.500000  14.000000
75%    15.750000  15.750000
max    18.000000  17.000000
```

Explicación

`head(n)` muestra las primeras n filas; `info()` muestra tipos de datos y valores no nulos; `describe()` da conteo, media, desviación, mínimo, cuartiles y máximo de las columnas numéricas.
Para ver un DataFrame de forma visual en VS Code, abre el archivo como cuaderno Jupyter (Paso 15) y usa el visor de **Variables > Data Viewer**.

Paso 12. Seleccionar, filtrar, agrupar y crear columnas

Crea **04_pandas_operaciones.py**. Aquí combinamos Pandas con `np.where` de NumPy para clasificar a los alumnos.

Archivo: 04_pandas_operaciones.py

```
# 04_pandas_operaciones.py - Selección, filtros, columnas nuevas y agrupación
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "alumno": ["Ana", "Luis", "Rosa", "Pedro", "Carla", "Jorge"],
    "carrera": ["Software", "Redes", "Software", "Redes", "Software", "Redes"],
    "nota1": [15, 12, 18, 10, 14, 16],
    "nota2": [17, 11, 16, 13, 12, 15],
})

# 1. Seleccionar columnas
print(df["alumno"].tolist())
print(df[["alumno", "nota1"]])

# 2. loc (por etiqueta) e iloc (por posición)
print(df.loc[0, "alumno"])          # fila 0, columna 'alumno'
print(df.iloc[1:3, 0:2])            # filas 1-2, columnas 0-1

# 3. Crear columnas calculadas
df["promedio"] = (df["nota1"] + df["nota2"]) / 2
df["estado"] = np.where(df["promedio"] >= 13, "Aprobado", "Desaprobado")
print(df)
```

```
# 4. Filtrar filas
print(df[df["promedio"] >= 15])
print(df[(df["carrera"] == "Redes") & (df["estado"] == "Aprobado")])

# 5. Ordenar
print(df.sort_values("promedio", ascending=False).head(3))

# 6. Agrupar y resumir
resumen = df.groupby("carrera")["promedio"].agg(["mean", "max", "count"])
print(resumen.round(2))
print(df["estado"].value_counts())
```

Resultado esperado en la terminal

```
['Ana', 'Luis', 'Rosa', 'Pedro', 'Carla', 'Jorge']
  alumno  nota1
0    Ana     15
1   Luis     12
2   Rosa     18
3  Pedro     10
4  Carla     14
5  Jorge     16
Ana
  alumno   carrera
1   Luis     Redes
2   Rosa  Software
  alumno   carrera  nota1  nota2  promedio       estado
0    Ana  Software     15     17      16.0     Aprobado
1   Luis     Redes     12     11      11.5  Desaprobado
2   Rosa  Software     18     16      17.0     Aprobado
3  Pedro     Redes     10     13      11.5  Desaprobado
4  Carla  Software     14     12      13.0     Aprobado
5  Jorge     Redes     16     15      15.5     Aprobado
  alumno   carrera  nota1  nota2  promedio    estado
0    Ana  Software     15     17      16.0  Aprobado
2   Rosa  Software     18     16      17.0  Aprobado
5  Jorge     Redes     16     15      15.5  Aprobado
  alumno carrera  nota1  nota2  promedio    estado
5  Jorge   Redes     16     15      15.5  Aprobado
  alumno   carrera  nota1  nota2  promedio    estado
2   Rosa  Software     18     16      17.0  Aprobado
0    Ana  Software     15     17      16.0  Aprobado
5  Jorge     Redes     16     15      15.5  Aprobado
           mean   max  count
carrera
Redes     12.83  15.5      3
Software  15.33  17.0      3
estado
Aprobado       4
Desaprobado    2
Name: count, dtype: int64
```

Operación	Sintaxis	Qué hace
Seleccionar columnas	`df[["a", "b"]]`	Devuelve un DataFrame solo con esas columnas.
Por etiqueta	`df.loc[fila, columna]`	Usa nombres de índice y columnas.

Operación	Sintaxis	Qué hace
Por posición	`df.iloc[0:3, 0:2]`	Usa números de posición (como en listas).
Filtrar	`df[df["x"] > 10]`	Conserva solo las filas que cumplen la condición.
Varias condiciones	`(cond1) & (cond2)`	Usa & (y), | (o). Cada condición entre paréntesis.
Ordenar	`df.sort_values("x", ascending=False)`	Ordena de mayor a menor.
Agrupar	`df.groupby("g")["x"].mean()`	Calcula un resumen por grupo.
Contar valores	`df["x"].value_counts()`	Cuenta cuántas veces aparece cada valor.

Paso 13. Leer un CSV, limpiar datos y exportar a Excel

Crea **05_pandas_archivos.py**. En proyectos reales los datos llegan con errores o celdas vacías (**NaN**). Este ejemplo crea un CSV de ventas, lo lee, completa los valores faltantes y genera un reporte.

Archivo: 05_pandas_archivos.py

```
# 05_pandas_archivos.py - Leer CSV, limpiar datos y exportar a Excel
import pandas as pd
import numpy as np

# 1. Crear un archivo CSV de ejemplo (simula datos reales)
ventas = pd.DataFrame({
    "fecha": ["2026-01-05", "2026-01-12", "2026-02-03", "2026-02-20", "2026-03-08",
"2026-03-15"],
    "producto": ["Laptop", "Mouse", "Laptop", "Teclado", "Mouse", "Teclado"],
    "cantidad": [2, 10, np.nan, 5, 8, 4],
    "precio": [2500, 35, 2500, 80, 35, np.nan],
})
ventas.to_csv("ventas.csv", index=False)
print("Archivo ventas.csv creado")

# 2. Leer el CSV
df = pd.read_csv("ventas.csv", parse_dates=["fecha"])
print(df)

# 3. Detectar y tratar valores faltantes (NaN)
print("Valores faltantes por columna:")
print(df.isna().sum())
df["cantidad"] = df["cantidad"].fillna(df["cantidad"].median())
df["precio"] = df["precio"].fillna(df.groupby("producto")["precio"].transform("mean"))
print(df)

# 4. Columnas derivadas
df["total"] = df["cantidad"] * df["precio"]
df["mes"] = df["fecha"].dt.month
print(df)

# 5. Tabla resumen por mes y producto
```

```
tabla = df.pivot_table(values="total", index="mes", columns="producto",
                       aggfunc="sum", fill_value=0)
print(tabla)

# 6. Exportar resultados
df.to_excel("reporte_ventas.xlsx", index=False)
print("Reporte exportado a reporte_ventas.xlsx")
```

Resultado esperado en la terminal

```
Archivo ventas.csv creado
       fecha producto  cantidad  precio
0 2026-01-05   Laptop       2.0  2500.0
1 2026-01-12    Mouse      10.0    35.0
2 2026-02-03   Laptop       NaN  2500.0
3 2026-02-20  Teclado       5.0    80.0
4 2026-03-08    Mouse       8.0    35.0
5 2026-03-15  Teclado       4.0     NaN
Valores faltantes por columna:
fecha       0
producto    0
cantidad    1
precio      1
dtype: int64
       fecha producto  cantidad  precio
0 2026-01-05   Laptop       2.0  2500.0
1 2026-01-12    Mouse      10.0    35.0
2 2026-02-03   Laptop       5.0  2500.0
3 2026-02-20  Teclado       5.0    80.0
4 2026-03-08    Mouse       8.0    35.0
5 2026-03-15  Teclado       4.0    80.0
       fecha producto  cantidad  precio    total  mes
0 2026-01-05   Laptop       2.0  2500.0   5000.0    1
1 2026-01-12    Mouse      10.0    35.0    350.0    1
2 2026-02-03   Laptop       5.0  2500.0  12500.0    2
3 2026-02-20  Teclado       5.0    80.0    400.0    2
4 2026-03-08    Mouse       8.0    35.0    280.0    3
5 2026-03-15  Teclado       4.0    80.0    320.0    3
producto   Laptop  Mouse  Teclado
mes
1          5000.0  350.0      0.0
2         12500.0    0.0    400.0
3             0.0  280.0    320.0
Reporte exportado a reporte_ventas.xlsx
```

Explicación

`isna().sum()` cuenta los valores faltantes por columna.
`fillna(mediana)` reemplaza los vacíos de **cantidad** con la mediana; para **precio** se usa el promedio del mismo producto con `groupby(...).transform("mean")`.
`pivot_table` crea una tabla dinámica como en Excel: meses en filas, productos en columnas y la suma de ventas en las celdas.
Revisa el Explorador de VS Code: aparecerán los archivos **ventas.csv** y **reporte_ventas.xlsx**. Haz clic en el CSV para verlo dentro del editor.

Parte 4. Visualización, cuadernos y depuración

Paso 14. Graficar los resultados con Matplotlib

Crea **06_grafico.py**. Al ejecutarlo se abrirá una ventana con el gráfico y se guardará la imagen **promedios.png** en la carpeta del proyecto.

Archivo: 06_grafico.py

```
# 06_grafico.py - Visualizar los resultados con Matplotlib
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "alumno": ["Ana", "Luis", "Rosa", "Pedro", "Carla", "Jorge"],
    "promedio": [16.0, 11.5, 17.0, 11.5, 13.0, 15.5],
})

colores = ["tab:green" if p >= 13 else "tab:red" for p in df["promedio"]]
plt.bar(df["alumno"], df["promedio"], color=colores)
plt.axhline(13, linestyle="--", color="gray", label="Nota mínima (13)")
plt.title("Promedio por alumno")
plt.ylabel("Promedio")
plt.legend()
plt.tight_layout()
plt.savefig("promedios.png")
plt.show()
```

Resultado esperado en la terminal

```
Se abre una ventana con un gráfico de barras: barras verdes para promedios ≥ 13,
barras rojas para promedios < 13 y una línea punteada gris en 13.
En el Explorador aparece el archivo promedios.png.
```

Paso 15. Trabajar con un cuaderno Jupyter en VS Code

Los cuadernos permiten ejecutar el código por celdas y ver tablas y gráficos debajo de cada una. Son muy usados en ciencia de datos.

1. Presiona `Ctrl + Shift + P` y elige **Crear: Nuevo Jupyter Notebook** (Create: New Jupyter Notebook).
2. Arriba a la derecha pulsa **Seleccionar kernel** y elige tu entorno **venv**. Si VS Code lo pide, acepta instalar el paquete **ipykernel**.
3. En la primera celda escribe `import pandas as pd` y `import numpy as np`; ejecútala con `Shift + Enter`.
4. En otra celda crea el DataFrame de alumnos y escribe solo `df` en la última línea: verás la tabla con formato.
5. Pulsa **Variables** en la barra superior del cuaderno y luego el ícono junto a `df` para abrir el **Data Viewer** (visor de datos), donde puedes ordenar y filtrar.
6. Guarda el cuaderno como **07_practica.ipynb** con `Ctrl + S`.

Usar el depurador de VS Code

El depurador te permite detener el programa en una línea y revisar el valor de cada variable. Es la mejor forma de entender qué hace el código paso a paso.

1. Haz clic a la izquierda del número de línea donde quieres detenerte: aparecerá un **punto rojo** (punto de interrupción). También puedes usar `F9`.
2. Presiona `F5` y elige **Python Debugger > Python File** (archivo actual).
3. El programa se detiene en el punto rojo. En el panel **VARIABLES** (izquierda) verás los arreglos, tensores o DataFrames con sus valores.
4. Usa `F10` para avanzar línea por línea, `F11` para entrar a una función y `Shift + F5` para detener.
5. En la **CONSOLA DE DEPURACIÓN** puedes escribir expresiones, por ejemplo `df.head()`, y ver el resultado en ese momento.

Parte 5. Errores frecuentes y cómo resolverlos

Mensaje de error	Causa	Solución
`No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...`	Python no está instalado o no está en el PATH; Windows redirige el comando a la Microsoft Store.	Instala Python desde python.org marcando **Add python.exe to PATH**. Desactiva los alias en **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** (python.exe y python3.exe). Cierra y vuelve a abrir VS Code.
`ModuleNotFoundError: No module named 'pandas'`	La librería no está instalada en el intérprete seleccionado.	Activa **venv**, selecciona su intérprete (Paso 6) y ejecuta `pip install pandas`.
`KeyError: 'Nota1'`	El nombre de columna no existe (mayúsculas, tildes o espacios).	Revisa con `print(df.columns)` y escribe el nombre exacto.
`ValueError: cannot reshape array of size 12 into shape (5,3)`	El total de elementos no coincide.	Usa dimensiones cuyo producto sea igual al tamaño (12 = 3×4 = 2×6).
`ValueError: operands could not be broadcast together`	Arreglos de tamaños incompatibles.	Verifica `shape` de ambos arreglos antes de operar.
`FileNotFoundError: ventas.csv`	La terminal está en otra carpeta.	Abre la carpeta del proyecto en VS Code o usa la ruta completa del archivo.
`ModuleNotFoundError: No module named 'openpyxl'`	Falta el motor para escribir Excel.	Ejecuta `pip install openpyxl`.

Parte 6. Ejercicios propuestos

1. **NumPy:** genera con `default_rng(seed=7)` un arreglo de 30 notas enteras entre 0 y 20. Muestra el promedio, la nota más alta, cuántos aprobaron (≥ 13) y el porcentaje de aprobados.
2. **NumPy:** crea una matriz 4×3 con `arange` y `reshape`. Calcula la suma por filas, el promedio por columnas y su transpuesta.
3. **Pandas:** crea un DataFrame con 8 productos (nombre, categoría, precio, stock). Agrega la columna `valor_inventario = precio * stock` y muestra los 3 productos con mayor valor.
4. **Pandas:** agrupa los productos por categoría y muestra el stock total y el precio promedio de cada una.
5. **Limpieza de datos:** a partir de `ventas.csv`, elimina las filas con datos faltantes usando `dropna()` y compara cuántas filas quedan frente a la versión con `fillna()`. Escribe en un comentario cuál método conviene y por qué.

6. **Reto:** descarga un dataset público en CSV (por ejemplo, de **datosabiertos.gob.pe**), cárgalo con Pandas, muestra `info()` y `describe()`, y elabora un gráfico de barras con un resumen agrupado.

Lista de cotejo (autoevaluación)

Criterio	Logrado	En proceso
Configuré el entorno virtual y seleccioné su intérprete en VS Code.	☐	☐
Creé arreglos con array, arange, linspace, zeros, ones y reshape.	☐	☐
Apliqué operaciones vectorizadas, estadísticas, máscaras y producto matricial.	☐	☐
Creé Series y DataFrames y los exploré con head, info y describe.	☐	☐
Seleccioné con loc/iloc, filtré, ordené y agrupé datos.	☐	☐
Leí un CSV, traté valores faltantes y exporté un reporte a Excel.	☐	☐
Elaboré un gráfico y usé el depurador o un cuaderno Jupyter en VS Code.	☐	☐